Comunicación asíncrona accesible

Matías Lira Breyan Sebastián

—

Desarrollo Web Profesional

—

José Miguel Carrera Pacheco

ASYNC JAVASCRIPT

Async JavaScript (JavaScript asíncrono) es una forma de programación que permite ejecutar tareas que tardan tiempo sin bloquear la ejecución del resto del programa.

En una aplicación web, algunas operaciones pueden tardar varios segundos, por ejemplo:

- consultar datos en un servidor
- cargar información desde una API
- descargar archivos
- realizar consultas a bases de datos

Si estas operaciones se ejecutaran de forma síncrona, la página web se congelaría mientras espera la respuesta.

Formas de trabajar con asincronía en JavaScript

Callbacks

Un callback es una función que se ejecuta después de que termina una operación.

Ejemplo:

```
function obtenerDatos(callback){
    setTimeout(function(){
        callback("Datos recibidos");
    }, 2000);
}
```

En este caso, la función se ejecuta después de 2 segundos.

Promesas (Promises)

Las promesas representan el resultado futuro de una operación.

Ejemplo:

```
fetch("https://api.ejemplo.com/datos")
.then(response => response.json())
.then(data => console.log(data));
```

Las promesas pueden tener tres estados:

- pendiente
- cumplida
- rechazada

Async / Await

Es una forma moderna de manejar operaciones asíncronas y facilita la lectura del código.

Ejemplo:

```
async function obtenerDatos(){
    const respuesta = await
fetch("https://api.ejemplo.com/datos");
    const datos = await respuesta.json();
    console.log(datos);
}
```

APIS REST

Una API REST (Representational State Transfer) es una interfaz que permite que diferentes aplicaciones se comuniquen entre sí a través de internet utilizando el protocolo HTTP.

Este tipo de arquitectura fue propuesto por Roy Fielding.

Las APIs REST se utilizan para intercambiar información entre cliente y servidor.

Métodos HTTP principales

Método	Función
GET	Obtener información
POST	Crear nuevos datos
PUT	Actualizar datos
DELETE	Eliminar datos

Ejemplo de solicitud a una API

Solicitud:

```
GET https://api.ejemplo.com/usuarios
```

Respuesta del servidor:

```
[
 { "id": 1, "nombre": "Ana" },
 { "id": 2, "nombre": "Luis" }
]
```

Las APIs REST son muy utilizadas en aplicaciones web modernas y frameworks de JavaScript.

ESTADOS DE CARGA ACCESIBLES

Los estados de carga accesibles son indicadores que informan al usuario que una acción está en proceso.

Esto ocurre cuando el sistema necesita tiempo para completar una tarea, por ejemplo:

- cargar información de un servidor
- enviar un formulario
- procesar datos

Si el sistema no muestra un estado de carga, el usuario puede pensar que la aplicación dejó de funcionar.

Ejemplos de estados de carga

- mensajes de texto como **"Cargando..."**
- animaciones de carga
- barras de progreso

Ejemplo:

```
<div>Cargando información...</div>
```

Accesibilidad en estados de carga

Para usuarios que utilizan lectores de pantalla se pueden usar atributos ARIA.

Ejemplo:

```
<div aria-live="polite">
Cargando información
</div>
```

Este atributo permite que los lectores de pantalla anuncien los cambios dinámicos en la página.

MANEJO DE ERRORES DE RED

El manejo de errores de red consiste en detectar y responder adecuadamente cuando una solicitud a internet falla.

Esto puede suceder por diversas razones:

- falta de conexión a internet
- problemas en el servidor
- tiempo de espera agotado
- errores en una API

Si la aplicación no maneja estos errores, el usuario puede quedarse sin respuesta o sin saber qué ocurrió.

Ejemplo en JavaScript

```
fetch("https://api.ejemplo.com/datos")
.then(response => {
    if(!response.ok){
        throw new Error("Error en la solicitud");
    }
    return response.json();
})
.then(data => console.log(data))
.catch(error => {
    console.log("Ocurrió un error:", error);
});
```

Buenas prácticas

1. Mostrar mensajes claros al usuario.
2. Permitir reintentar la operación.
3. Detectar si no hay conexión a internet.
4. Registrar errores para depuración del sistema.

Ejemplo de mensaje para el usuario:

No se pudo conectar al servidor. Intenta nuevamente más tarde.